

EECE7352 Computer Architecture

Homework 1

Sakthivel P. Sivakumar

NUID: 002312294

Part F:

1) Why does Amdahl's Law need to be extended, according to Hill and Marty?

According to the authors (Hill and Marty), Amdahl's law needs to be extended because the computing industry has entered the "Multicore Era" where **performance gains come from increasing the number of processors on a single chip**. The original law provides a model for the software speedup based on the fraction of the code that can be parallelized, but it **doesn't account for the new hardware trade-offs** that chip designers face.

Designers must now decide how to use a chip's finite resources (**area & power**), and they should also think about either to create many simple, small cores (or) fewer, more powerful, and complex cores.

They should also think about how this choice affect performance for programs with both sequential and parallel components.

To address this, the author introduces a corollary to Amdahl's Law that incorporates a **hardware cost model**. This model, based on a "**base core equivalent**" (BCE), allows for the analysis of trade-offs between making a single core faster (which helps the sequential part of a program) versus adding more cores (parallel part). In summary, the **extension is needed** to make Amdahl's Law **relevant to the physical design constraints** and choices of modern multicore processors.

2) Why do issues of multicore symmetry impact Amdahl's Law?

The issue in multicore symmetry impacts Amdahl's Law because it controls how a chip's resources are allocated to handle a program's sequential and parallel sections, leading to different speedup calculations. The paper addresses this with two main architectures:

Symmetric Multicore Chips: -

All cores on the chip are identical in performance and cost. If a chip's resource budget is used to create a few powerful cores, the total number of available cores decreases. During the sequential part of a program, only one of these powerful cores is used. During the parallel part, all of the identical cores work together. This creates a direct trade-off: making cores more powerful helps the sequential phase but hurts the parallel phase by reducing the core count. The **formula for symmetric speedup** reflects this balance:

$$\text{Speedup}_{\text{symmetric}}(f, n, r) = \frac{1}{\frac{1-f}{\text{perf}(r)} + \frac{f \cdot r}{\text{perf}(r) \cdot n}}$$

Asymmetric Multicore Chips: -

The chip contains cores of different capabilities, typically **one powerful core and multiple simpler, standard cores**. The single powerful core is dedicated to executing the sequential code, providing a significant boost. During the parallel phase, the powerful core works alongside all the simpler cores. This approach avoids the harsh trade-off of the symmetric model, as it can accelerate the sequential bottleneck without sacrificing as many total cores for parallel execution. The **formula for Asymmetric speedup** is

$$\text{Speedup}_{\text{asymmetric}}(f, n, r) = \frac{1}{\frac{1-f}{\text{perf}(r)} + \frac{f}{\text{perf}(r) + n - r}}$$

Therefore, **symmetry is a crucial factor** because **it defines the processor's capability during both the serial (1-f) and parallel (f) portions of execution**, directly altering the performance terms within the Amdahl's Law equation.

3) How can a dynamic multicore chip offer greater speedup compared to an asymmetric chip?

A dynamic multicore chip can offer greater speedup than an asymmetric chip because **it is designed to reconfigure its resources optimally** for both sequential and parallel modes,

- Sequential - fusing smaller cores into big one
- Parallel – Splitting the fused cores for parallel execution

authors describe this concept as having "**their cake and eat it too**". The key difference lies in how each chip handles the parallel execution phase:

- An asymmetric chip has a fixed design: one large core (with resources r) and $n-r$ smaller cores. In the parallel part of a program, its total performance is the sum of the large core and the small cores: $\text{perf}(r) + (n-r)$.
- A dynamic multicore chip is more flexible. For the sequential part of a program, it can dynamically "gang" the resources of multiple simple cores together to create one powerful, virtual core with performance $\text{perf}(r)$. However, for the parallel part, it can release these resources, allowing all n base cores to operate independently. This yields a total parallel performance of n .

Since the performance of a powerful core is typically less than its resource cost (i.e., $\text{perf}(r) < r$), the parallel performance of the dynamic chip (n) is greater than that of the asymmetric chip ($\text{perf}(r) + n - r$). **The dynamic chip gets the sequential benefit of a large core without compromising on having the maximum number of cores available for parallel tasks, allowing it to achieve a higher overall speedup.**